Security Challenges in the Implementation of Artificial Intelligence in System-on-Chip (SoC) and Network-on-Chip (NoC) Architectures

1st Manjot Singh Dhillon
Department of Computer Science
The University of Sheffield
Sheffield, United Kingdom
manjotdhillon0144@outlook.com

2nd Harshada Manoj Gavade
Department of Computer Science
The University of Sheffield
Sheffield, United Kingdom
harshadagawade411@gmail.com

3rd Syed Hamad Zubair Gilani
Department of Computer Science
The University of Sheffield
Sheffield, United Kingdom
hamadgilani@live.com

4th Bhagawan Sekhara Raju
Department of Computer Science
The University of Sheffield
Sheffield, United Kingdom
bsraju1@sheffield.ac.uk

***Abstract*—Modern AI accelerators and on-chip learning engines are built on System-on-Chips (SoC) and Network-on-Chip (NoC) fabrics. This extends the attack surface and puts a load on standard security primitives. In this paper, we review a range of on-chip threats including fault injection, side-channel attacks, eavesdropping, traffic analysis, Denial-of-Service (DoS), and adversarial/backdoor attacks. The paper also examines state-of-the-art defences including Lightweight encryption and anonymous routing, Deep-Q-Learning (DQL) control policies, adversarial-robust training and run-time anomaly detection. Finally, reviewing the challenges from previous research paper, found that solutions primarily tackled single layers within the SoC and NoC architectures, resolving vulnerabilities within hardware, network, or AI layer. Furthermore, examination of open challenges proposed future research to point towards securing all layers of the evolving SoC/NoC architectures to safeguard against advancing threats, while maintaining a minimal performance and area overhead.**

***Index Terms*—System-on-Chip, Network-on-Chip, Artificial Intelligence, fault injection, side-channel attacks, eavesdropping, traffic analysis, Denial-of-Service, adversarial/backdoor attacks, lightweight encryption, anonymous routing, Deep-Q-Learning control policies, adversarially robust training, run-time anomaly detection**

I. Introduction

Nowadays, digital systems come with enhanced performance, integration and are more efficient in terms of energy. The System-on-Chip (SoC) has become the primary choice for embedding complex functionality. They range from general-purpose cores and hardware to memory controllers and I/O interfaces. Many high-end SoCs include a Network-on-Chip (NoC), a modular interconnect fabric that replaces conventional buses and provides low-latency, high-bandwidth communication across a diverse range of parts. These advancements provide unparalleled processing capacity; however, they significantly increase the on-chip attack surface.

In this paper, we analyse recent research conducted on the security challenges confronting SoC and NoC architectures in the context of Artificial Intelligence (AI), along with the mitigation and defence mechanisms proposed for on-chip systems.

The following section delves into prior literature on the technologies referred to within this paper, specifically System-on-Chip, Network-on-Chip, AI Integration in SoC/NoC, and Security Foundations.

A. *System-on-Chip Architectures*

System-on-Chips (SoCs) are Integrated Circuits (ICs) that integrate an end-product's essential components within a single chipset or chip. This architecture includes a Central Processing Unit (CPU), memory, I/O ports, and peripheral interfaces [1]. According to [2], "Modern SoCs can incorporate billions of transistors within a compact area of around one hundred square millimetres." This technology allows SoCs to play an integral part in modern technology and electronics.

*1) **Components of SoC Architectures**:*

- **Processing Units** include CPUs, Graphics Processing Units (GPUs), and AI accelerators.
- **Memory Systems** include DRAM or SRAM modules that are tightly coupled with processors to form a high-bandwidth memory pool, minimising latency [5].
- **Interconnect Fabrics**: modern SoCs employ Network-on-Chip (NoC) architectures, whereas legacy designs rely on bus-based connections.
- **Peripheral and Interfaces**: these include USB, Ethernet, PCIe, cryptographic engines, and Hardware Roots of Trust (HWRoT).

SoCs often contain multiple third-party Intellectual Property (IP) cores, modular functional blocks that implement specific

tasks like signal processing, encryption, or AI inference. While these enhance the board's functionality and speed up the manufacturing process, they also introduce a range of diverse security vulnerabilities under real usage [3]. To reduce development costs, many IP blocks are outsourced, which requires sharing entire design specifications; this leads to introducing attack vectors, including intellectual property theft, counterfeiting, and reverse engineering [4].

B. Network-on-Chip Architectures

Network-on-Chip (NoC) is an important part of multicore computer processors that enable data transfer between cores and memory units while improving data transfer efficiency and scalability [11].

According to [35], "NoC handles wiring delays and congestion much better than even a small 5x5 crossbar component". This feature enables better scalability and design feasibility in modern SoCs.

*1) **Components of NoC Architectures** [35]:*

- **Switches**: Routes packets between sources and destinations and manages congestion.
- **Links**: Connects Network Interfaces (NIs) and switches.
- **Network Interfaces (NIs)**: Connects IP cores to the NoC and converts transactions into packets.

While NoCs improve performance, energy efficiency, and scalability, they are becoming more vulnerable to security threats like **Hardware Trojans (HTs)**. This is a small, malicious circuit inserted into the hardware by untrusted third parties. HTs can leak or corrupt sensitive data crossing the NoC. These vulnerabilities often go undetected during modern verification and testing processes, raising serious hardware trust issues [35].

C. AI Integrations in SoC/NoC

Integrating **Artificial Intelligence (AI)** into SoCs offers ways to enhance the systems' functionality, performance and security. AI accelerators enable compelling applications within compact devices that allow efficient execution of AI workloads, offering advantages over regular CPUs. Neural Processing Units, such as hardware accelerators designed to meet the demands of Deep Neural Network (DNN) based Machine Learning (ML) algorithms [6]. An example of an ultra-low power AI accelerator is the dual-core MCU (Arm Cortex-M4 and RISC-V) and a Convolutional Neural Network (CNN) accelerator used in the MAX78000 [7], enabling efficient on-device AI processing. Modern NPUs support parallelism of multiple tasks, introducing high-level resource management challenges and security vulnerabilities [8]. Furthermore, **Large Language Models (LLMs)** are a revolutionising technology that can enhance SoC security through automated vulnerability detection and policy generation frameworks. An implementation of the LLM-powered SoC security framework is the DIVAS framework, which provides an automated end-to-end solution that accepts user specifications for generic SoCs and uses LLMs to identify relevant Common Weakness Enumerations (CWEs) [9]. DIVAS subsequently converts these CWEs into SystemVerilog Assertions (SVAs), using LLM models like GPT and BARD, for verification and generates security policies that can be implemented through centralised security models [9]. While implementing AI enhances to overall SoC/NoC systems, it is likely to introduce new attack vectors and security concerns that must be addressed through security measures.

D. Security Fundamentals

Due to the focus of the paper, it is important to understand how the security of a system is established on three basic principles: confidentiality, integrity and availability [32].

- **Confidentiality** protects sensitive information from unauthorised access.
- **Integrity** ensures that data is correct and unaltered throughout transmission and storage.
- **Availability** guarantees that authorised users have a reliable and timely access to resources.

A systematic approach, known as threat modelling, is used to identify security concerns, specify attacker capabilities and categorise probable assaults. It is an important basis for a strong security taxonomy and defence strategies across different system layers.

II. Methodology

A. Search Timeline

Our search for peer-reviewed papers was carried out between 31 March 2025 and 16 May 2025.

B. Data Sources

For the development of this Systematisation of Knowledge the following repositories and publication sites have been used to capture both peer-reviewed and emerging work:

- IEEE Xplore
- ACM Digital Library
- arXiv
- USENIX Security Symposiums
- Network and Distributed System Security (NDSS) Symposiums
- European Symposium on Research in Computer Security (ESORICS)
- Pseudo-Pets Conference (PoPETs)

Combining existing work from these resources, ensures the coverage of established work.

C. Search Strategy

In order to achieve a complete coverage of relevant literature on AI security threats in SoC and NoC architecture, a systematic search strategy was employed:

*1) **Keyword Selection** :* Search was conducted on critical keywords that address both AI threats as well as hardware security in the context of SoC/NoC. Some of the keywords employed are:

- "AI in System-on-Chip"
- "Security in Network-on-Chip"

- ”AI vulnerabilities in hardware”
- ”Threat models for AI and SoC”
- ”Hardware-based security in NoC/SoC”

*2) **Search Methodology**:* The search strategy was meant to reveal both core research and cutting-edge research in the field.

- Initial searches involved browsing titles, abstracts, and keywords for pertinence to hardware AI vulnerabilities and SoC/NoC security.
- A broader search was conducted to find the latest studies, particularly those published in recent times, i.e., within the last five years, to accumulate the latest data.
- Citations of influential papers were also browsed to find further pertinent studies.

*3) **Selection Criteria**:*

- Inclusion: Peer-reviewed articles, conference proceedings, and papers dealing with AI security in SoC/NoC architectures were included.
- Exclusion: Papers that were not directly connected to the central theme of the work and those that were not published on the previously stated publication sites were excluded.

*4) **Review Process**:* After the end-point identification, articles were carefully read to screen for appropriateness with regards to the objectives of the Systematisation of Knowledge (SoK). The final selection was aimed at presenting a balanced view of theoretical frameworks and practical applications of security in AI-driven SoC/NoC systems.

III. Threat Model

A threat model recognises the potential adversaries, assumptions, and capabilities about which system components can be trusted. It also outlines the critical assets which require protection, such as data, system configurations, and model parameters. The attacker's goal may seek to achieve data exfiltration, system disruption, or unauthorised access. By systematically analysing these elements, the threat model helps evaluate system resilience against a wide range of attack vectors and design effective security mechanisms [10].

*1) **Adversary Capabilities**:* Physical access offers intrusive probing, fault injection, or reverse engineering of SoC internals. Non-intrusive attacks include eavesdropping on NoC messages and exploiting side channels such as power, electromagnetic radiation, or timing variations to extract sensitive information. The attackers may also exploit firmware-level vulnerabilities or supply chain attacks to inject hardware Trojans [10].

*2) **Trust Assumptions**:* The SoC architecture is assumed to run in partial trust. Even though secure boot and trusted execution environments (TEEs) are used for isolation, some of the NoC components (e.g., routers, links) and third-party intellectual property (IP) blocks are not trusted in their entirety. Moreover, the on-chip NoC communication is typically unencrypted due to latency and resource constraints, making it a viable candidate for observation and manipulation [10].

*3) **Critical Assets**:* The majority of critical assets at risk are:

- When made public, model weights used by AI accelerators leak intellectual property and make adversarial attacks easier [12].
- On-chip inference or sensor data, potentially including privacy-sensitive or proprietary data.
- Cryptographic keys and system scheduling or task allocation metadata can be used to reverse engineer system behaviour or cause Denial-of-Service (DoS) states [10].

*4) **Attacker Goals**:* The attacker may have various motivations, including:

- Intellectual property theft through model extraction [12].
- Data exfiltration through passive monitoring of on-chip traffic.
- System Denial-of-Service or degradation, particularly traffic flooding or contention against NoC routers [11].
- Privilege escalation or control flow hijack, through exploitation of configuration register or firmware-execution flows [10].

These capabilities and motivations emphasise the need for a strong, multi-layered security posture across physical, architectural, and runtime defences.

A. Hardware-level Threats

Threats to SoC and NoC architectures can be segregated into subsections. One of these is attacks directed towards the hardware components of SoC/NoC.

*1) **Fault Injection**:* These are active physical attacks that are used for malicious purposes. It can extract cryptographic keys, privilege escalation, and attacks on neural network implementations [12]. Multiple fault injection mechanisms exist, such as [30]:

- **Voltage Glitching** is a well-known attack technique that can cause faulty behaviour in processors during operation. Glitching the supply voltage can corrupt the data values, skip security checks, or enable protected code paths. This can destabilise a processor's operation and cause it to skip instructions, miscompute logic, or bypass security checks. [24]
- **Clock Glitching**: It is a timing-based fault injection technique in which an attacker can inject an anomalous pulse into the system clock to disrupt the timing of circuit operations. This exploits most synchronous digital circuits as it relies on precise clock edges to trigger transitions and logic evaluations. The reference [30] has mainly focused on the effects of clock glitching on D-type flip flops, which are fundamental building blocks in any digital circuits, including those found in NoC and SoC architectures.
- **Laser Fault Injection**: Laser Fault Injection is the most commonly used technique as it involves directing a focused laser beam at specific areas of the semiconductor device to induce faults. This technique can cause

flipping of single bits or altering logic states, especially in sensitive parts like activation functions within neural networks. Research studies show that targeting activation functions such as SoftMax, ReLU, sigmoid, and tanh in deep networks can lead to misclassification, highlighting the vulnerability of AI models to such attacks. [30]

*2) **Side-Channel Attacks**:* It is a type of security vulnerability that exploits non-functional system behaviour, such as power consumption, electromagnetic emissions, heat, and acoustic waveforms, which are not part of the data flow but still attack a secure system. [33]

As a Secure system often takes different times to perform different operations, this variation is exploited by **Timing Side Channels** to extract and leak sensitive data. For example, Reinbrecht, C. [36] demonstrated a practical Prime with a Probe timing attack on a NoC-based SoC. As the target was communication between an ARM Cortex-A9 and shared cache memory, this was done in 4 steps:

1) **Infection**: Malware enters into the SoC system.
2) **Prime**: Attacker overwrites certain areas of the shared cache to clear the previous Advanced Encryption Standard (AES) lookup table.
3) **Probe**: Attacker monitors cache behaviour and its location during the execution.
4) **Analysis**: These cache patterns reveal information about the secret keys.

Other studies have carried out timing attacks, where attackers infect cores to inject traffic and observe network latency, revealing secure communication timing and patterns. Wang et al in their work showed in the timing attacks can inter with Rivest-Shamir-Adleman (RSA) Key by observing network traffic which was caused by modular applications [33].

B. Network-level Threats in NoC

Network-level attacks in NoC architectures are critical security attacks that exploit communication patterns and resource vulnerabilities. These are:

*1) **Eavesdropping and Traffic Analysis**:* Eavesdropping refers to the unauthorised monitoring and interception of communications without any alterations. This often targets metadata such as packet timing, source, destination, size, and routing information rather than the actual payload contents in digital systems. Encryption protects the data, but traffic analysis can exploit side channels to leak communication patterns and reveal sensitive information. By analysing the transmitted packets size, frequency and timing, the attacker can deduce private systems behaviour such as AI inference operations, inter-core communications, and memory access sequences [13]. Major threats posed by eavesdropping and traffic analysis:

- **Confidentiality Violations**: Attackers can inactively observe communication flows to extract sensitive information, even when data payloads are encrypted [13].
- **Intellectual Property Theft**: Attackers can gather AI models, proprietary algorithms, or business secrets embedded in the SoC by interpreting traffic patterns [12], [13].
- **Exposure of Sensitive Training Data**: Machine learning accelerators on NoC can leak user input types or critical training data through detectable traffic patterns [12].
- **System Profiling for Future Attacks**: Adversaries can create detailed profiles of system activity, making future active attacks (e.g., fault injections or targeted DoS attacks) more feasible [10], [13].

Steps of eavesdropping and traffic analysis attack:

1) **Initial access**: Attackers usually begin by identifying vulnerable points in a network, such as unsecured Wi-Fi or weakly configured security measures [10].
2) **Passive monitoring**: Observes and analyses the network traffic secretly without altering or interrupting communication.
3) **Traffic collection**: An attacker intercepts network traffic to reveal sensitive information or network topology, which can then be used for attacks like jamming or spoofing.
4) **Feature extraction**: It involves extracting and identifying characteristic data from NoC traffic that potentially indicates patterns of malicious activity, such as eavesdropping [13].
5) **Inference and profiling**: It involves using collected traffic data to infer the nature of the communication, such as data type, application, or even sensitive information, by analysing patterns and anomalies in the traffic stream [12], [13].
6) **Exploitation**: Attacks exploit vulnerabilities to modify, intercept, or analyse on-chip communication, to reveal sensitive data or compromise system integrity [10], [13].

*2) **Denial-of-Service (DoS)**:* A DoS attack aims to disrupt a system's normal functioning by making critical components such as cores, memory units, and peripheral devices unavailable to its intended users. The attacker aims to reduce the system's performance and efficiency, and ensure it is completely stopped. [13] These attacks are particularly detrimental in NoC/SoC systems, as they can slow down the network transmission and consume the bandwidth. [13] This type of attack reaches particular relevance in embedded systems, where the availability of resources is restricted to the users and mainly reduces the capabilities of the inter-core communication within the infrastructure. [15] Regarding bandwidth reduction in DoS attacks, it ensures that IPS Communicate with other network resources, which causes a higher latency on the chip transmission. Due to this, the system fails to complete certain operations within the desired time frame. [33] As we know, NoC is the backbone of SoC, facilitating efficient data exchanges between numerous processing cores and hardware blocks. Any disruption to NoC can affect SoC's overall performance and reduce the ability of communication

in the infrastructure, which may be compromised [14], [15]. The following are the mechanisms for DoS attacks:

- **Deadlock**: This occurs in NoC when packets are introduced into the network, which can cause a circular dependency concerning the routing technique. In such a state, the packet fails to reach its desired destination and gets stuck at some intermediate resource, blocked from being used for other transmissions. This blockage can lead to a standstill, where no further data transmission can occur, disrupting system operations. [15]
- **Packet flooding**: The attacker tries to flood the network with a large volume of packets, exhausting the network's capacity to handle the traffic. This, in turn, leads to congestion, such as network congestion and buffer overflow [15].
- **Routing Manipulation**: A vulnerability in NoC's routing algorithms can cause packets to be routed or blocked, resulting in data delays or packet loss. This vulnerability can cause certain parts of the systems to be disconnected, and the ability to communicate between Processing Elements (PEs) will be lost [15].
- **Packet corruption**: Repeatedly sending corrupted packets can also lead to a DoS attack. HTs tamper with the input buffer when entering a router leading to performance degradation and packet loss, wasting resources on retransmissions, buffer space, and response delays [15].

C. *AI-specific Vulnerabilities*

The integration of AI poses vulnerabilities specific to it that, in conjunction with SoC and NoC, develop new attack vectors.

*1) **Adversarial Examples**:* Adversarial attacks are threats targeting neural networks. They inject deceptive input into AI models to make incorrect predictions that are not perceptible to human observers. **Black-box adversarial attacks** are one type in which the attacker cannot access the AI model's structure or the training dataset, with only the ability to send an input and receive the model's output back. There are three main types of black-box attacks: non-adaptive black-box, adaptive black-box, and strict black-box attacks [21].

- In **non-adaptive black-box attacks**, only the training data distribution is known to the attacker, but the interaction with the model is limited [21].
- In **adaptive black-box attacks**, the attacker can interact with the model freely with various inputs and observe the outputs generated [21].
- In **strict black-box attacks**, access is highly restricted, with a limitation to legitimate samples only [21].

This is a significant vulnerability for SoCs using compact ML models and NoCs, which enable parallel processing on model computations. Their ability to freely run queries makes them easy targets for black-box adversarial attacks. In addition, NoCs do not authenticate the input they receive from the model; instead, they trust the input without any anomaly detection.

*2) **Backdoor Attacks**:* A backdoor attack is an exploit where the attacker secretly manipulates a ML model to inject malicious behaviour when triggered by specific input [22]. An example of a backdoor attack is the DeepPayload attack described by [23], during which the attacker decompiles the model to a data flow graph format, and a malicious payload that monitors inputs for the trigger is added to the graph. Then this payload actively awaits the trigger, until the trigger is detected, the model operates normally, making it harder to notice it [23]. This presents a vulnerability for SoCs, as if the models are deployed without appropriate model verification. An attacker could reverse-engineer the model and inject the backdoor payload, replacing the model file on the SoC storage. Since the SoC does not verify the model's integrity, it unknowingly loads the tampered model.

IV. Taxonomy of Security Challenges

A. *Taxonomy of Threats*

Previous research tackles the problems of attack vectors or design layers. Possible solutions miss the fact that SoC/NoC designs have to cover three main areas:

1) **Hardware (HW)**: This layer is the primary attack surface for fault-injection, side-channel attacks and Trojan insertion.
2) **Network (Net)**: This has become the main target of attackers through passive observation and low-effort active attacks.
3) **AI**: The AI layer introduces newer threats that merge into hardware vulnerabilities.

The table below shows how the main attack classes target each layer and which asset is at risk.

TABLE I
Attack Classes and Assets at Risk

Attack Class	HW	Net	AI	Asset at Risk
Fault injection	✓	-	-	Execution integrity
Side-channel	✓	✓	✓	Model parameters
Traffic analysis	-	✓	-	Data confidentiality
DoS	✓	✓	-	Availability
HTs / back doors	✓	✓	✓	Design integrity
Adversarial attacks	-	-	✓	Model accuracy
IP theft	✓	✓	✓	IP ownership
Cross-layer attacks	✓	✓	✓	Mixed

Figure 1 illustrates a formalised attack taxonomy of vectors against AI integrated SoC and NoC design structures. Attacks are grouped from three perspectives: By Intent, By Target, and By Technique.

- **By Intent**: Attacks are either Active-involving direct manipulation in the form of fault injection, denial of service, and adversarial examples, or Passive-such as side-channel attacks and eavesdropping, which observe without disrupting function.
- **By Target**: Tags the vulnerable components like AI accelerators, NoC routers, memory blocks, and sensors.

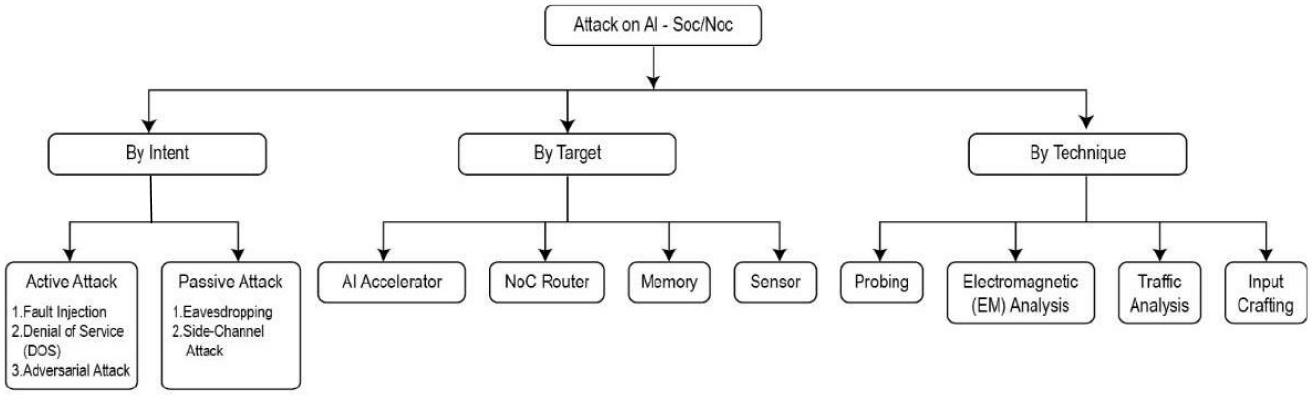

Fig. 1. Taxonomy of Attacks on AI-SoC/NoC

- **By Technique**: Refers to specific approaches like physical probing, electromagnetic (EM) analysis, observation of traffic patterns, and input crafting.

Such categorisation enables one to comprehend the broad diversity of threats and leads the security mechanisms' design being particular to those threats.

B. *Key insights from the Taxonomy*

Reviewed research papers provided a comprehensive description of security vulnerabilities mitigation models. Although their focus was on a system providing overall security, they failed to address some key challenges mentioned in this section.

1) *Gaps in Protection:*

- Faults in the NoC control logic are basically an open attack surface. Due to vulnerabilities with buffer management, packets can be dropped or rerouted without errors. Previous research has not suggested any hardware hardening of the NoC control.
- The issue of adversarial attacks delivered through on-chip networks is completely ignored. None of the research has looked into this direction, with the majority presuming a software injection.
- Trojan-induced manipulation in the NoC remains an unexplored area.

2) *Lack of Innovation:*

- Three reviewed papers have reinvented timing masking for traffic confidentiality, each suggesting minor alterations between themselves.
- At the hardware accelerator level, techniques to mitigate side-channel attacks replicated cryptographic design methodologies while ignoring unique AI characteristics.

3) *Failure to Cross Layers:*

- Research papers introduce mitigation techniques that focus on protecting hardware, networks, or AI models separately. None of the surveyed work focuses on providing an end-to-end attack chain countermeasure.
- This is a central underdeveloped area, considering attackers already use cross-layer attacks.

A general overview of a cross-layer attack could be processed as follows:

1) NoC routing logic fault introduced by clock glitching.
2) Packet misrouting leads to the leakage of DNN input features through side channel attacks.
3) Adversarial perturbations are introduced during AI inference, leading to misclassification.

V. Mitigation & Defence Mechanisms

In recent years, the integration of AI has introduced several security challenges requiring newer, robust hardware and software-based defence mechanisms.

A. *Hardware-based Defences*

As AI advances and its embedding within SoC and NoC designs improves, this technology generates new vulnerabilities previously unthought of. Modern systems increasingly depend on AI accelerators, that allow deep learning models to be improved. However, these models are vulnerable to side channelling attacks, which could potentially compromise the information processed by AI accelerators.

Research [14] describes a specialised hardware-based defence mechanism utilising **Dual-Rail Precharge Logic (DPL)** for DNN systolic array safeguarding against power analysis attacks. Within this design, each processing element has been given two engines, a **multiply-accumulate (MAC)** and a **complementary MAC (CMAC)**, where each cell is cloned and inverted from the MAC. Before each clock tick, a beat keeps the whole system in rhythm. MAC and CMAC are switched and reset to a precharge stage, where they are off, "00". Half a tick later, they switch to the actual code, where one switch is on and the other is off, "10" or "01", followed by

the reset, then flip, ensuring that the same amount of energy is used at all times regardless of the data inside. Every processing element comprises two identical rows of memory cells driven by two control signals: active1 and active2. These signals work in turns, while the row under active1 is in reset mode, the row under active2 executes. After each half-tick, jobs are swapped until the job is fully executed, then an OR gate joins their output into a single stream to ensure that the array sees a single stream of numbers. Due to each gate utilising the exact reset and flip cycle, the energy utilised by the chip remains stable, leading to power analysis attacks not being able to examine power usage to determine whether a 0 or 1 was processed lowering correlation coefficient to 0.045 during the analysis of 5000 power traces, indicating significant improvements against side channel security within AI accelerators.

Figure 2 showcases the difference between the power consumption of a Single-rail logic circuit, DR logic circuit, and DPL logic circuit, as it can be seen the DPL logic circuit maintains a steady energy output, when compared to the other circuits, effectively showing the difference between previous models and the one presented by [14].

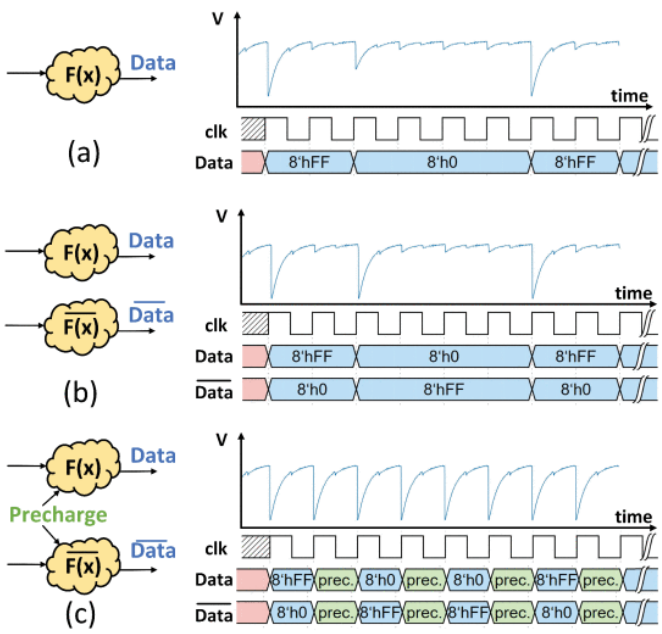

Fig. 2. Schematic of the power consumption [14]

B. Secure NoC Design Techniques

Since NoC has become a standard component in SoC designs, it has become a primary target for security attacks. Hence, the main goal has become to ensure the confidentiality, integrity and authenticity of on-chip communications with minimal performance and area overhead. Below are a few solutions that have been proposed:

*1) **Lightweight Encryption and Anonymous Routing (LEARN) Protocol:*** According to [17], in resource-constrained embedded systems, typical encryption and authentication methods are ineffective for security. For this reason, Charles and Mishra [17] propose *LEARN*, a framework that utilises addition and multiplication operations, instead of complex cryptographic operations, for lightweight encryption that is based on secret sharing and polynomial interpolation. The framework also utilizes anonymous routing so that intermediate nodes fail to identify where a packet is coming from or where it is headed. LEARN consists of two main phases [17]:

- **Phase 1** – Route Discovery: During this phase, a three-way handshake (consisting of Route Initiate, Accept and Confirmation) is performed between the sender and receiver. The network path is found by sending packets and assigning random nonces and a hidden polynomial to each router.
- **Phase 2** – Data Transfer: This phase uses the parameters assigned to each router to send the original message securely and anonymously through the network. Each router only sees those local nonces (so it cannot tell where packets came from or where they are going) and changes the message using its polynomial piece, ensuring that only the final destination can fully decrypt and reassemble the message. This approach makes the content safe from eavesdropping attackers. Figure 3 shows an overview of the LEARN framework.

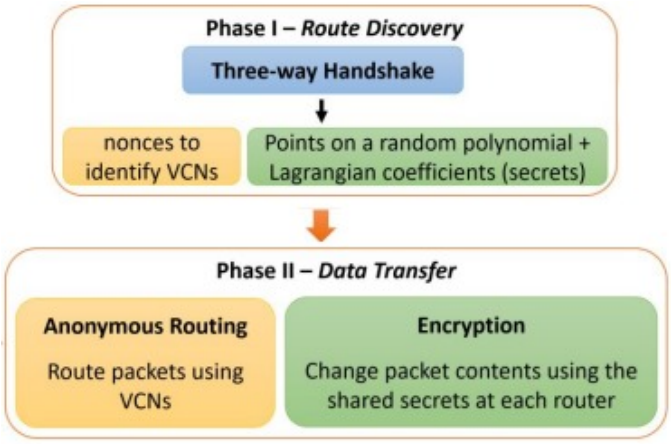

Fig. 3. LEARN framework overview [17]

TABLE II
LEARN PERFORMANCE EVALUATION

Metric	Value
NoC delay overhead	+28%
Execution time overhead	+4%
Improvement over encryption & AR	+74% (NoC delay), +30% (execution time)
Area overhead	$\sim$ 20 kB

The table above shows that the LEARN model successfully achieves encryption and anonymous routing with an increase in **4%** execution time and **28%** NoC delay compared to

an insecure baseline. The model also outperforms traditional encryption and routing methods by **74%**in NoC delay and an increase in execution time by **30%**. LEARN skips the 1.5x performance degradation as it achieves near baseline performance, costing only ~ 20 **kB** of area overhead.

*2) **Pre-Router Threat Detection Hardware**:* Wang and Li [34] propose a per-router threat detection hardware which facilitates a fully connected artificial neural network, named D-ANN, in their *SecureNoC* framework. The neural network is made up of an input layer, hidden layer and a output layer. The purpose of this detection system is to identify abnormal behaviour induced by Hardware Trojans (HT). D-ANN learns from runtime network activities and automatically detects HT infected routers by monitoring and extracting the attribute values (buffer and link utilisation, packet injection rate and local operation temperature) of local runtime from the on-chip sensors. These values are fed into the input layer of D-ANN and are used to calculate a label indicating whether the HT is infected or free.

*3) **Deep-Q-Learning (DQL)-Based Control Policy**:* Part of Wang and Li's *SecureNoC* framework [34] that adds hardware and algorithmic defences to a NoC chip, DQL is a type of reinforcement learning algorithm that learns from the interactions between routers and NoC systems. It uses a DNN to approximate a Q-function and selects the highest value action at each control interval. The goal here is to learn which combinations of security methods (bypass links, lightweight encryption, etc.) provides the best balance throughput and energy. *SecureNoc* leverages a DQL control policy to manage its security mechanisms in response to changing network conditions. DQL policy can adapt to new traffic patterns or attack profiles, invoking defences only when they're most needed, minimising performance and power penalties. Figure 4 shows the Deep-Q-Learning process.

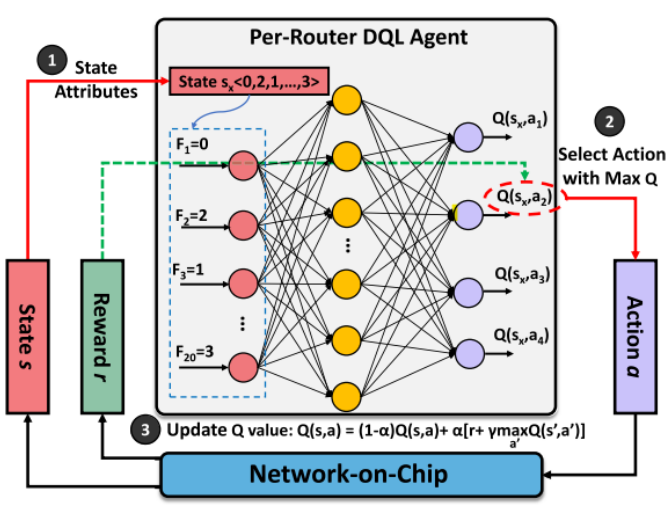

Fig. 4. Deep-Q-Learning process [34]

C. *AI-specific Security Solutions*

AI is a technology that contains vulnerabilities even before its incorporation within SoCs and NoCs. This section aims to provide some solutions to AI-specific vulnerabilities.

*1) **Adversarial-robust training**:* Quantised Neural Networks (QNNs) are essential for efficiently deploying AI on hardware platforms. However, these models showcase lower robustness against adversarial attacks, which involve minimal perturbations to input data to compromise the output classification. Therefore adversarial training is required to increase robustness, this is achieved within the research [16] by adding tampered images into the standard training process, using **Fast Gradient Sign Method (FGSM)** to make minimal changes to the input image adding noise to trick the model using three perturbation levels: $\epsilon = 0.01, 0.02, 0.03$. Because QNNs use very low precision (4-bit weights and 2-bit activation), the adversarial changes are also rounded to fit within the same 2-bit activation level, making the attack seem realistic for the operations of the model on hardware. During this training process, the model receives the normal and tampered images in each batch, learning to adjust its 4-bit weights to classify the attacked inputs appropriately. This is all achieved through the use of **Straight-Through Estimator (STE)** to allow gradients to pass through quantisation steps during backpropagation, with only the weight file to be updated on the **Field-Programmable Gate Array (FPGA)** without requiring changes on the hardware structure.

TABLE III
ACCURACY IMPROVEMENTS

Performance evaluation		
	Before	***After***
Clean Data	90%	92.5%
White-box attacks	66%	81%
Black-box attacks	84%	88%

In the end, the model described by [16]'s adversarial training improved its performance against adversarial attacks to increase its average accuracy without increasing its resource usage. Table 3 showcases the improvements in average model accuracies.

*2) **Run-time anomaly detection in AI accelerators**:* Research by [19] proposes a self-powered sensing system utilising *TinyML* for anomaly detection on the device, eliminating heavy reliance on cloud servers, cutting energy usage up to 66.74% and reducing risks related to raw data transmission. This prototype substitutes **Inertial Measurement Units (IMUs)** with a piezoelectric cantilever sensor. An IMU is an electronic device that provides motion measurements independently of environmental factors [20]. The newer sensor captures the machine's vibrations to generate voltage, without requiring an extra power supply. These vibrations are also sampled at 20 Hz, optimising energy usage. Moreover, on the sensor side, there is no heavy preprocessing. A compressed DNN is trained on this noisy data. *TensorFlow Lite* com-

presses the model through quantisation and pruning, making it small enough to fit within a microcontroller within a SoC. This microcontroller samples intermittently and infers locally, transmitting only when an anomaly is detected. This model achieved an anomaly detection with 97.6% accuracy while saving energy compared to traditional setups [19].

VI. Case Studies

Within this section some case studies are analysed for commercial SoC/NoC devices in production.

A. *Commercial AI SoCs*

Tegra X1 and X2 SoCs are designed for embedded systems requiring high-performance computing and AI capabilities. The NVIDIA Tegra X2 is a powerful SoC utilising a Pascal-family GPU architecture with 8GB of memory optimised for AI workloads. Work by [24] revealed a significant vulnerability in in the NVIDIA Tegra X2, where the researchers used voltage glitching to physically attack the chip, temporarily destabilising its power supply during start-up, causing the re-enabling of an inaccessible debugging feature under normal circumstances. This backdoor allowed the researchers to run their code with the highest privilege level. Once they gained access, researchers dumped the *BootROM*, which included critical encryption keys that are burned into the chip by the factory. These keys allow potential bad actors to decrypt and reverse engineer the rest of the chip start-up process and firmware, completely disrupting the secure boot protections. This attack was conducted with "only cheap and readily available equipment," taking between two and ten seconds [24]. The NVIDIA Tegra X2 is not the only exploitable chip. The Tegra X1 was exploited in similar ways, leading to the Nintendo Switch jailbreak in 2017. Given that the two attacks follow the same pattern in two different generational SoCs, safety concerns arise. The researchers, working with NVIDIA, disclosed possible measures to mitigate this exploit, including hardening the physical board to prevent access to sensitive voltage rails. However, these are temporary solutions; effective solutions would require changes in hardware and software design [24].

B. *Research Prototype*

A prototype has been developed by some researchers from the University of Florida to counter reverse engineering threats in SoC designs, especially those making use of NoC communications, by developing **Obfuscated Network-on-Chip (ObNoCs)**, which is an obfuscation framework tailored for NoCs, and introduces programmable switches concealing the real communication topology during untrusted phases of the supply chain [25]. The central concept involves the replacement of the static router connections with programmable switches that allow multiple plausible topologies, significantly decreasing the chances for an attacker to manipulate the board structure within the assembly stage without access to the activation package, which is loaded later during the process reconfiguring the switches to restore communication pathways as they were intended [25].

VII. Open Challenges & Future Directions

The rise in AI-based features in SoC and NoC architectures raise several security concerns that are still under explored. Below we outline key open issues, drawing on recent literature to highlight gaps and future research directions.

A. *Growing NoC Techniques (Electronic, Wireless and Photonic)*

Depending on how a NoC is implemented, whether electronic, photonic or wireless, it remains vulnerable to a large attack surface that can be exploited, corrupted and disrupt data transfers [31].

Open challenges for **electronic NoCs** include having a security framework that involves an overhead around all NoC routers. This causes performance bottlenecks. In addition, many HT detection systems include message authentication and cryptographic schemes. These can disrupt the path of NoC routers, leading to poor performance and overheads. **Wireless NoCs** include the transceivers and their performance under process variations, making establishing countermeasures for security threats challenging. Furthermore, the shared nature of wireless channels makes them more vulnerable to attacks. Photonic NoCs include designing and detecting HTs with silicon photonic devices (e.g., MRs, photodetectors, etc.). **Photonic NoCs** are very sensitive to runtime temperature fluctuations and fabrication process differences. This makes it critical to investigate how such variations affect attack detection and mitigation strategies [31].

B. *Scalability of Security Solutions*

Integrating SoC and NoC design paradigms with AI poses significant scalability problems for security designs. While the platforms are made more complex to support practical high-performance AI computations, conventional security methods fall behind in updating without sacrificing noticeable performance overhead. This is an acute problem for latency-constrained or power-sensitive environments such as embedded or edge AI applications.

One of the significant concerns is the performance-security trade-off. Standard encryption, secure routing, and intrusion detection approaches will likely add processing delay and increase power consumption. In [17], it is mentioned that existing NoC security mechanisms may cause serious performance degeneration. They proposed a lightweight routing and anonymous encryption scheme designed particularly for high-security performance in low cost NoC-based SoCs, emphasising the necessity of scalable and low-footprint techniques. Moreover, the diverse nature of modern SoCs comprising CPUs, GPUs, AI accelerators, and custom hardware Ips makes it challenging to implement security uniformly. Every unit has distinct threat surfaces and resource needs, making the one-size-fits-all security model inappropriate. In addition, the dynamic execution of AI workloads introduces novel considerations to the threat profile. AI inference and training workloads can dynamically shift execution patterns during runtime, so security measures capable of real-time response are needed.

In [27], employing LLMs to support SoC security verification would enable scalable, intelligent frameworks to detect and solve security problems in real-time dynamically. Finally, the absence of normalised tests and testing procedures impedes the comparative assessment of proposed security measures. Without consistent measurements and testbeds, scalability and robustness in novel approaches become difficult to test over different usage profiles and system configurations. In summary, scalable security for SoC/NoC design with AI empowerment needs root solutions that are lightweight, adaptive, and hardware-conscious. Fulfilling this challenge will also equate to standard test environments and interdisciplinarity that unites hardware design, ML, and cybersecurity expertise.

C. *Standardisation & Benchmarks*

Integrating AI into SoC and NoC architectures introduces new and complex attack surfaces that existing security frameworks struggle to model accurately. Recent work has highlighted the need for more asset-centric threat modelling approaches to protect critical AI components (e.g., model weights, training datasets, inference outputs) across distributed or embedded hardware environments. For example, a 2025 study proposes an asset-driven methodology for AI security that explicitly addresses vulnerabilities across the AI system lifecycle, revealing that current frameworks miss crucial cross-layer threats in hardware-software co-designs [28].

In addition, the absence of standardised metrics to measure the security posture of AI-integrated SoCs and NoCs significantly hinders advancements in this domain. It is challenging to compare security mechanisms or measure their strength under real-world constraints without standardised metrics or test cases. This leads to a fragmented research landscape where techniques are often validated only in simulations or highly controlled environments.

To address this, we propose the development of SoC-Bench, a modular, open-source benchmark suite designed to evaluate cross-layer security across hardware, network, and AI domains. This benchmark would include synthetic AI workloads, attack scripts for fault injection, traffic flooding and eavesdropping, and adversarial robustness testing. Evaluation metrics should span execution integrity, model accuracy under attack, NoC latency, packet loss, and energy overhead. The benchmark should be containerised to enable reproducibility and cross-platform deployment. The field remains fragmented and non-comparable without a standard, reproducible framework for validating security claims. SoC-Bench would provide a standardised, lifecycle-aware foundation for measuring, comparing, and improving security across emerging AI-integrated SoC/NoC systems.

Emerging tools that leverage LLMs for automating threat modelling processes have shown early promise. For instance, the Auspex framework explores how LLMs can assist in building security threat models by guiding developers through structured questions and generating model-specific threat analyses [29]. While promising, such AI-assisted tools are still nascent and lack formal validation against known AI hardware threat cases.

In summary, addressing this challenge requires community-wide efforts to develop AI-specific threat models and benchmarking suites that are hardware-aware, reproducible, and scalable. Without these, the secure design and deployment of AI-SoC/NoC platforms will remain inconsistent and vulnerable to emerging attack vectors.

D. *Interdisciplinary Approaches*

Analysis of our current research on securing Artificial Intelligence (AI) on SoC and NoC Architecture often focuses on isolated domains like hardware-level protections, AI robustness, or secure interconnects. However, the real malicious actors exploit any technology available to them to carry out their attack, leading to them moving between the layers. Therefore, a hardware-level defence can only guard against one type of attack, exposing the other areas. Attackers exploit weaknesses in AI models using hardware-level access (e.g., Fault Injection to influence the outputs of neural networks) or to compromise the interconnects to extract models' sensitive data, as this fragmentation in research leaves systems vulnerable to coordinated, cross-layer attacks.

To address this, Interdisciplinary approaches are essential. One example is the DIVAS framework [9], which deploys large language Models (LLMs) like GPT and BARD to automate SoC security verification. By translating Common Weakness Enumerations (CWEs) into System Verilog Assertions, DIVAS connects AI/NLP capabilities with formal hardware verification—bridging AI and electronic design automation (EDA). Similarly, LLM4SoC also proposes a paradigm where AI models assist hardware engineers in dynamic threat models and security policy generation [27], showcasing the energy between natural language processing, machine learning, and hardware security.

Another example is the Secure-SoC framework [34], which integrates a Deep-Q Learning (DQL) policy into the NoC router logic. This engages the reinforcement learning model, dynamically adjusting security mechanisms based on traffic conditions and threat patterns, demonstrating an effective fusion of AI and NoC runtime controls. Also, adversarial-robust quantised neural networks (QNNs) on FPGAs [16] not only showcase the necessity of co-design: model training techniques must be optimised for hardware constraints like 4-bit weights and activation clipping. This approach combines AI adversarial training, embedded system constraints, and FPGA-level deployment.

Furthermore, the Auspex framework [29] extends interdisciplinary work into the human-in-the-loop domain by using LLM-based assistants to help security engineers develop structured threat models for AI-integrated systems. This blends human factors, software security, AI, and hardware design principles.

TABLE IV
GAP ANALYSIS

Area Evaluated	Current Status	Gaps Identified
Threat Modelling for AI-SoC/NoC	Emerging efforts, but lacking asset-centric and cross-layer focus	No full-stack threat models
Benchmarks for AI-integrated SoCs/NoCs	Non-existent. Validations mostly simulation based	No real world reproducible benchmarking
LLM Assisted Threat Modelling	Early stage and promising	Tools not tested in real environments
Evaluation of Security Mechanisms	Fragmented across hardware, network and AI separately	No integrated evaluation across layers

These examples illustrate the need for collaborative research that spans multiple fields: hardware architecture, AI/ML, formal methods, and cybersecurity. Without such integration, defences remain siloed and inadequate. Future work should prioritise end-to-end, cross-layer security models co-designed by experts across disciplines to ensure scalable and resilient protection for AI-SoC/NoC systems.

E. Future Directions

AI is deeply embedded within SoC and NoC architectures, and end-to-end and adaptive security are crucial. As we know, traditional methods are not meeting the unique demands of today's AI hardware environment, as performance and scalability pressure increase. One of the most important future directions is the development of lean, scalable, and AI-aware security mechanisms. With SoCs embracing growing numbers of cores and AI accelerators, the overhead of conventional security protocols is unaffordable. Research such as the DIVAS framework demonstrates how scalable policy enforcement can be made possible through intelligent automation, using LLMs to analyse and protect against vulnerabilities during the design phase [9]. Similarly, a lightweight encryption and anonymous routing was proposed for NoC-based SoCs, to ensure secure communication with less resource utilisation like in edge or real-time systems [17]. Another necessity is the development of standardised benchmarks and evaluation environments. Most of these security measures are still tested in simulated or synthetic settings, limiting their applicability to real-world scenarios. Using open-source testbeds and threat models promoted by recent surveys can improve reproducibility, comparability, and deployment readiness of security controls for AI-driven designs [2], [26]. Finally, the field must focus on embracing cross-layer, cross-domain security techniques. Most current defences assault the AI models (e.g., adversarial robustness) or the hardware (e.g., hardware Trojan detection), but real attackers typically exploit vulnerabilities across multiple layers. Solutions like LLM-for-SoC security indicate a compelling direction where software wisdom complements hardware security, incorporating co-designed, end-to-end perspectives on securing AI-fueled platforms [9], [27].

Collaborative research that meets machine learning, hardware architecture, and security engineering knowledge will be the foundation of future advancement. Ultimately, context-aware, dynamic, and scalable security architectures grounded in theory and practice will be required to protect the next generation of intelligent computing systems.

VIII. CONCLUSION

This SoK paper presents a comprehensive analysis of security threats arising from the implementation of AI in SoC and NoC architectures. The threats have been categorised into hardware-level vulnerabilities, network-level vulnerabilities, and AI-specific vulnerabilities, and we have proposed a structured threat model taxonomy spanning fault injection and side channel attacks to adversarial examples and hardware backdoors. Our study demonstrates that the increasing complexity of SoC/NoC designs, especially with AI accelerators, has significantly broadened the attack surface. The taxonomy developed here maps threats systematically across hardware, network, and AI layers. However, future work must extend this taxonomy to capture lifecycle dimensions, classifying vulnerabilities by their emergence phase: pre-silicon, manufacturing, and post-deployment. Understanding when a threat emerges enables targeted, stage-specific mitigations. We have surveyed a range of mitigation strategies, including Lightweight Encryption and Anonymous Routing protocols (LEARN), DQL reinforcement learning for dynamic NoC reconfiguration, adversarial robust training of AI models, and TinyML-driven real-time anomaly detection. Despite these advances, significant research gaps persist. Current devices often lack scalability, fail to generalise across diverse NoC types, and are evaluated based on inconsistent benchmarks. This leads to most solutions only protecting one layer rather than addressing the cross-layer nature of modern attacks. Case studies on commercial AI SoCs such as NVIDIA Tegra X1/X2 and experimental platforms like obNoC highlight that commercial systems remain vulnerable to physical and structural attacks, suggesting that robust, scalable, and real-world tested defences are urgently needed.

In conclusion, this paper emphasises the importance of a unified SoK approach in guiding future research and development. As AI becomes increasingly embedded in hardware platforms, fostering collaboration among hardware engineers, AI developers, and cybersecurity experts is essential for building secure, trustworthy, and robust system architectures.

ACKNOWLEDGMENT

The group would like to thank the course instructor for their guidance and constructive feedback throughout the project. We also acknowledge the use of Overleaf, IEEE Xplore, and various AI and security research resources, which were instrumental in the successful completion of this work.

REFERENCES

[1] Martin, G. and Chang, H., 2001, October. System-on-Chip design. In ASICON 2001. 2001 4th International Conference on ASIC Proceedings (Cat. No. 01TH8549) (pp. 12-17). IEEE.

[2] Kibria, R., Farahmandi, F. and Tehranipoor, M., 2024. A Survey on SoC Security Verification Methods at the Pre-silicon Stage. Cryptology ePrint Archive.

[3] Farzana, N., Rahman, F., Tehranipoor, M. and Farahmandi, F., 2019, November. Soc security verification using property checking. In 2019 IEEE International Test Conference (ITC) (pp. 1-10). IEEE.

[4] Rahman, M.M.M., Rahman, M.S., Kibria, R., Borza, M., Reddy, B., Cron, A., Rahman, F., Tehranipoor, M. and Farahmandi, F., 2023, April. CAPEC: A cellular automata guided FSM-based IP authentication scheme. In 2023 IEEE 41st VLSI Test Symposium (VTS) (pp. 1-8). IEEE.

[5] Hübner, P., Hu, A., Peng, I. and Markidis, S., 2025. Apple vs. Oranges: Evaluating the Apple Silicon M-Series SoCs for HPC Performance and Efficiency. arXiv preprint arXiv:2502.05317.

[6] Choi, Y. and Rhu, M., 2020, February. Prema: A predictive multi-task scheduling algorithm for preemptible neural processing units. In 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA) (pp. 220-233). IEEE.

[7] Min, C., Acer, U.G., Jang, S., Choi, S., Vasile, D.A., Gong, T., Yi, J. and Kawsar, F., 2024. An AI-Native Runtime for Multi-Wearable Environments. arXiv preprint arXiv:2403.17863.

[8] Feng, E., Feng, D., Du, D., Xia, Y. and Chen, H., 2024, June. sNPU: Trusted Execution Environments on Integrated NPUs. In 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA) (pp. 708-723). IEEE.

[9] Paria, S., Dasgupta, A. and Bhunia, S., 2023. Divas: An llm-based end-to-end framework for soc security analysis and policy-based protection. arXiv preprint arXiv:2308.06932.

[10] D. Lee, M. Tehranipoor, and S. Bhunia, "A Comprehensive Framework for Secure SoC Design," IEEE Design & Test, vol. 35, no. 5, pp. 30–38, Oct. 2018.

[11] Biglari, S., Hosseini, F., Upadhyay, A. and Zhao, H., 2024, December. Survey of Network-on-Chip (NoC) for Heterogeneous Multicore Systems. In 2024 IEEE 17th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC) (pp. 155-162). IEEE.

[12] A. Ramesh, M. Zheng, and A. Juels, "A Survey of Secure Architectures for AI Accelerators," ACM Computing Surveys, vol. 56, no. 2, Article 37, Feb. 2024.

[13] Charles, S. and Mishra, P., 2021. A survey of network-on-chip security attacks and countermeasures. ACM Computing Surveys (CSUR), 54(5), pp.1-36.

[14] Wu, L., Wu, L., Zhang, X. and Chinbat, M., 2024. Dual-Rail Precharge Logic-Based Side-Channel Countermeasure for DNN Systolic Array. IEEE Transactions on Very Large Scale Integration (VLSI) Systems.

[15] Fiorin, L., Palermo, G., Silvano, C. and Sami, M., 2009. Security in Networks-on-Chips. NETWORKS-ON-CHIPS, p.123.

[16] Lu, Y., Shi, X., Jiang, J., Deng, H., Wang, Y., Lu, J. and Wu, D., 2024. FPGA Adaptive Neural Network Quantization for Adversarial Image Attack Defense. IEEE Transactions on Industrial Informatics.

[17] Charles, S. and Mishra, P., 2023. Lightweight Encryption and Anonymous Routing in NoC based SoCs. arXiv preprint arXiv:2302.06118.

[18] Wang, H. and Halak, B., 2023, January. Hardware trojan detection and high-precision localization in noc-based mpsoc using machine learning. In Proceedings of the 28th Asia and South Pacific Design Automation Conference (pp. 516-521).

[19] Chen, Z., Gao, Y. and Liang, J., 2023, July. A self-powered sensing system with embedded tinyml for anomaly detection. In 2023 IEEE 3rd International Conference on Industrial Electronics for Sustainable Energy Systems (IESES) (pp. 1-6). IEEE.

[20] Khorrambakht, R., Lu, C.X., Damirchi, H., Chen, Z. and Li, Z., 2021. Deep Inertial Odometry with Accurate IMU Preintegration. arXiv preprint arXiv:2101.07061.

[21] Zanddizari, H., Zeinali, B. and Chang, J.M., 2021. Generating black-box adversarial examples in sparse domain. IEEE Transactions on Emerging Topics in Computational Intelligence, 6(4), pp.795-804.

[22] Guo, W., Tondi, B. and Barni, M., 2022. An overview of backdoor attacks against deep neural networks and possible defences. IEEE Open Journal of Signal Processing, 3, pp.261-287.

[23] Li, Y., Hua, J., Wang, H., Chen, C. and Liu, Y., 2021, May. Deeppayload: Black-box backdoor attack on deep learning models through neural payload injection. In 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE) (pp. 263-274). IEEE.

[24] Bittner, O., Krachenfels, T., Galauner, A. and Seifert, J.P., 2021, September. The forgotten threat of voltage glitching: a case study on Nvidia Tegra X2 SoCs. In 2021 Workshop on Fault Detection and Tolerance in Cryptography (FDTC) (pp. 86-97). IEEE.

[25] Halder, D., Merugu, M. and Ray, S., 2023. Obnocs: Protecting network-on-chip fabrics against reverse-engineering attacks. ACM Transactions on Embedded Computing Systems, 22(5s), pp.1-21.

[26] Oseni, A., Moustafa, N., Janicke, H., Liu, P., Tari, Z., & Vasilakos, A. (2021). Security and Privacy for Artificial Intelligence: Opportunities and Challenges. arXiv preprint arXiv:2102.04661.

[27] Saha, D., Tarek, S., Yahyaei, K., Saha, S. K., Zhou, J., Tehranipoor, M., & Farahmandi, F. (2023). LLM for SoC Security: A Paradigm Shift. arXiv preprint arXiv:2310.06046.

[28] S. Maheshwari, S. Osia, C. Ugochukwu, H. Karumanchi, J. Berrios, A. Huang, A. Ravichandran, and H. Deng, "Threat modeling for AI: The case for an asset-centric approach," arXiv preprint arXiv:2505.06315, 2025.

[29] J. Dethlefs, T. Semeniuk, K. Thomas, M. K. Reiter, and N. Christin, "Auspex: Building threat modeling tradecraft into an artificial intelligence copilot," arXiv preprint arXiv:2503.09586, 2025.

[30] Breier, J. and Hou, X., 2022. How practical are fault injection attacks, really?. IEEE Access, 10, pp.113122-113130.

[31] Pasricha, S., Jose, J. and Deb, S., 2022. Electronic, Wireless, and Photonic Network-on-Chip Security: Challenges and Countermeasures. IEEE Des. Test, 39(6), pp.90-98.

[32] Ray, S., Peeters, E., Tehranipoor, M.M. and Bhunia, S., 2017. System-on-chip platform security assurance: Architecture and validation. Proceedings of the IEEE, 106(1), pp.21-37.

[33] Y. Wang and G. E. Suh, "Efficient timing channel protection for on-chip networks," in Proc. 6th IEEE/ACM Int. Symp. Networks-on-Chip (NoC), 2012, pp. 142–151.

[34] Wang, K., Zheng, H., Li, Y. and Louri, A., 2021. SecureNoC: A learning-enabled, high-performance, energy-efficient, and secure on-chip communication framework design. IEEE Transactions on Sustainable Computing, 7(3), pp.709-723.

[35] Angiolini, F., Meloni, P., Carta, S., Benini, L. and Raffo, L., 2006, March. Contrasting a NoC and a traditional interconnect fabric with layout awareness. In Proceedings of the Design Automation & Test in Europe Conference (Vol. 1, pp. 1-6). IEEE.

[36] Reinbrecht, C., Susin, A., Bossuet, L., Sigl, G., & Sepúlveda, J. (2017). Timing attack on NoC-based systems: Prime+Probe attack and NoC-based protection. Microprocessors and Microsystems, 52, 556–565

APPENDIX

The work on this Systematisation of Knowledge paper has been split between four members of group 6. A WhatsApp group has been created to stay updated on the work being carried out by other group members. Due to the ease of doing so, meetings were also arranged through WhatsApp, and notifications were sent regarding modifications to be done and finishing tasks. All the information and formatting was completed on Overleaf using the IEEE template set up for Group 6. Moreover, the comments feature on Overleaf was also utilised to give feedback on the work done by colleagues.

A. Contributions

TABLE V
WEEKLY CONTRIBUTIONS OF EACH TEAM MEMBER

Team Member	Week	Tasks	Done
Manjot		Acted as group leader, overseeing tasks assigned and setting deadlines	✓
	2	Created an overall structure to follow for the SoK paper	✓
	3	Worked on background research on System-on-Chip architectures and AI integrations in SoC/NoC	✓
	4	Worked on AI specific vulnerabilities within the threat models' section	✓
	5	Worked on Mitigation and & Defence mechanisms, specifically completed hardware based and AI specific specific solutions	✓
	6	Worked on Taxonomy section	✓
	7	Worked on case studies	✓
	8	Proofread all sections to ensure no grammatical errors were present, along with rephrasing contents to ensure clarity	✓
	9	Created presentation slides for "Open Challenges and Future Directions" section and added upon the Conclusion. Ensured proper formatting of SoK paper and presentation slides	✓
Harshada	2	Worked on Methodology Section	✓
	3	Worked on Taxonomy of Security Challenge's section	✓
	5	Worked on Eavesdropping and Traffic Analysis in Network-level Threats in NoC	✓
	6	Created Figure 1 (Taxonomy of Attacks on AI-SoC/NoC) and edited Figure 3 (LEARN framework overview)	✓
	7	Worked on Open Challenges Section	✓
	8	Completed Future Directions	✓
	9	Worked on Mitigation and Defence Mechanisms	✓
	9	Created presentation slides for "Mitigation and Defence Mechanisms" section	✓
Syed	2	Conducted background research on Network-on-Chip architectures and AI integrations in SoC/NoC	✓
	3	Worked on Abstract section	✓
	4	Worked on introduction including the main introduction, Overview of NoC and Security Fundamentals	✓
	5	Worked on Methodology section	✓
	6	Worked on Taxonomy of Security Challenge's section	✓
	7	Conducted research and worked Mitigation and Defence Mechanism's section "Secure NoC Design Techniques"	✓
	8	Conducted research on open challenges and worked on Open Challenges section	✓
	9	Created template for presentation	✓
	9	Worked on "Introduction and Taxonomy of Security Challenges in the presentation	✓
	9	Ensured proper formatting of presentation slides and SoK paper	✓
Bhagawan	2–3	Conducted research and worked on Hardware-level Threats	✓
	4–6	Conducted research and worked on Denial-of-Service on Network-level threats in NoC	✓
	7-8	Conducted research and worked on Interdisciplinary Approaches	✓
	9	Worked on the Conclusion in the paper and Threat Model in the presentation	✓

*1) **Challenges Encountered**:* The primary challenge during the completion of this SoK paper was an initial misalignment in expectations and inconsistent updates during the first week working on the paper. This was instantly resolved by setting a weekly meeting schedule to check on tasks completed and ensure everyone was made aware of the work they were currently undertaking.

Secondly, at the initial phases of the project, there was an uneven workload distribution due to some members starting to work on new sections once their assigned workload had been completed, leading to the unavailability of work to be completed by other members.

Another challenge during this project was using different English vocabularies, US and UK, which led to the SoK paper being half with a UK and the other half with a US vocabulary, which was mutually changed to UK due to the University's geographical location.

*2) **Lessons Learnt**:* Learnt lessons for collaborative academic writing include that each individual has a separate skill set for working on diverse parts of the paper, which needs to be channelled in the appropriate direction to generate high-level academic writing.

*3) **Meetings**:* The table 6 contains the dates and key discussions of each meeting.

TABLE VI
MEETINGS LOG

Date	Discussion	Virtual / in-person?	Duration	Manjot	Harshada	Syed	Bhagawan
March 26, 2025	Initial meeting to discuss SoK topic selection. Short-listed two topics: "Intrusion Detection and Response in Embedded and Control Environments" and "Security Challenges in System-on-Chip (SoC) and Network-on-Chip (NoC) Architectures"	In-Person	1hr	✓	✓	✓	✓
April 3, 2025	Refined SoK topic, the initial scope for "Security Challenges in System-on-Chip (SoC) and Network-on-Chip (NoC) Architectures" was too broad therefore we finalised "Security Challenges in the Implementation of Artificial Intelligence in System-on-Chip (SoC) and Network-on-Chip (NoC) Architectures"	Virtual	30min	✓	✓	✓	✓
April 10, 2025	Weekly meeting: Initial meeting to define scope of the report and divide the initial work	Virtual	2hrs	✓	✓	✓	✓
April 17, 2025	Weekly meeting: Checking progress with assigned workload, with brief extensions for works still due, and assigning of new tasks for completed work	Virtual	1hr	✓	✓	✓	✓
April 24, 2025	Weekly meeting: Checking progress with assigned workload, with brief extensions for works still due, and assigning of new tasks for completed work	Virtual	1hr	✓	✓	✓	✓
May 1, 2025	Weekly meeting: Checking progress with assigned workload, with brief extensions for works still due, and assigning of new tasks for completed work	Virtual	1hr	✓	✓	✓	✓
May 8, 2025	Weekly meeting: Cross checking all inputted work, proofreading, and fixing mistakes throughout the SoK paper, along with changing any error individuated	Virtual	1hr	✓	✓	✓	✓
May 14, 2025	Review of entire SoK paper while making changes along the way	Virtual	4hrs	✓	✓	✓	✓
May 16, 2025	Final check: all tasks done; SoK & slides completed	Virtual	1hr 30min	✓	✓	✓	✓